CB

CODERBLUEPRINT

JAVA · SPRING BOOT · SYSTEM DESIGN

Design Patterns
Complete Interview Guide

All 23 Gang-of-Four patterns across Creational, Structural & Behavioral categories —
full definitions, real-world examples, Java implementations, UML, Spring Boot use cases, trade-offs, mistakes and interview Q&A.

23 Patterns | **23 Java Examples** | **UML Diagrams** | **Interview Q&A** | **Senior Tips**

What's Inside

Every pattern gets a dedicated page with the same interview-focused structure.

CREATIONAL PATTERNS

01 **Singleton** Pg. 3
02 **Factory Method** Pg. 4
03 **Abstract Factory** Pg. 5
04 **Builder** Pg. 6
05 **Prototype** Pg. 7

STRUCTURAL PATTERNS

01 **Adapter** Pg. 8
02 **Bridge** Pg. 9
03 **Composite** Pg. 10
04 **Decorator** Pg. 11
05 **Facade** Pg. 12
06 **Flyweight** Pg. 13
07 **Proxy** Pg. 14

BEHAVIORAL PATTERNS

01 **Strategy** Pg. 15
02 **Observer** Pg. 16
03 **Command** Pg. 17
04 **State** Pg. 18
05 **Template Method** Pg. 19
06 **Chain of Responsibility** Pg. 20
07 **Iterator** Pg. 21
08 **Mediator** Pg. 22
09 **Memento** Pg. 23
10 **Visitor** Pg. 24
11 **Interpreter** Pg. 25

What's Included?

- ✔ All 23 Design Patterns
- ✔ Divided into 3 Categories — Creational, Structural, Behavioral
- ✔ Easy-to-understand explanations
- ✔ Real-world examples
- ✔ Java implementation examples
- ✔ UML diagrams (where required)
- ✔ When to use & when NOT to use each pattern
- ✔ Advantages & disadvantages
- ✔ Common mistakes developers make
- ✔ Difference between similar design patterns
- ✔ Spring Boot & real project use cases
- ✔ Important interview questions for every pattern
- ✔ Senior-level interview discussion points

01 Singleton

Creational

Definition: Ensures a class has exactly one instance across the entire application and provides a single, well-known global access point to it.

Real-world use case: Configuration managers, connection pools, logging services, caches — anything that must never be duplicated in memory.

Spring Boot / real project: Every Spring @Component, @Service and @Repository bean is singleton-scoped by default inside the ApplicationContext. You almost never hand-roll the classic double-checked-locking Singleton in a Spring Boot codebase — you let the IoC container own the single instance and inject it wherever it's needed.

UML Diagram



How to read it: A single class exposes a private constructor and a static getInstance() access point — no other class can construct it directly.

Java Example

JAVA

```java
public class DatabaseConnectionManager {
    private static volatile DatabaseConnectionManager instance;

    private DatabaseConnectionManager() {
        // initialize connection pool
    }

    public static DatabaseConnectionManager getInstance() {
        if (instance == null) {
            synchronized (DatabaseConnectionManager.class) {
                if (instance == null) {
                    instance = new DatabaseConnectionManager();
                }
            }
        }
        return instance;
    }

    public void query(String sql) {
        System.out.println("Executing: " + sql);
    }
}

// Usage
DatabaseConnectionManager.getInstance().query("SELECT * FROM users");
```

WHEN TO USE

- Exactly one instance must coordinate actions across the whole app (config, cache, connection pool)
- You need lazy, thread-safe creation of an expensive shared resource
- A global point of access is genuinely required, not just convenient

WHEN NOT TO USE

- When the 'single instance' requirement is really just laziness about passing a dependency around
- In code you plan to unit test heavily — Singletons hide dependencies and make mocking hard
- In distributed / multi-JVM systems, where 'one instance' has to mean one per process, not truly global

ADVANTAGES

- Guarantees a single, consistent instance and controlled access to it
- Lazy initialization saves resources until the instance is actually needed
- Convenient global access point without passing references everywhere

DISADVANTAGES

- Introduces global mutable state, which makes reasoning about the program harder
- Hurts testability — hard to substitute a mock/stub for a hard-coded singleton
- Can silently become a concurrency bottleneck if synchronization is overused

COMMON MISTAKES

- Forgetting volatile on the instance field when using double-checked locking, which breaks under the Java Memory Model
- Making the class Serializable without a readResolve() method, which allows deserialization to create a second instance
- Reaching for Singleton when a Spring-managed bean (already effectively a singleton) would be simpler and more testable

DIFFERENCE: VS STATIC UTILITY CLASS

A Singleton is a real object with state and identity that can implement interfaces and be passed around; a static utility class is just a namespace for stateless functions. If you never need polymorphism or lazy state, a static class is simpler.

IMPORTANT INTERVIEW QUESTIONS

Q. How would you break a naive Singleton?
A. Reflection (calling the private constructor via setAccessible(true)), Java serialization (deserializing creates a new instance unless readResolve() is defined), and cloning if Cloneable is implemented without guarding clone().

Q. What's the safest way to write a Singleton in modern Java?
A. An enum with a single constant. Enums are inherently serialization-safe, reflection-safe against most attacks, and thread-safe by JVM class-loading guarantees — Joshua Bloch's recommended approach in Effective Java.

SENIOR-LEVEL DISCUSSION

In Spring, prefer letting the container manage singleton scope via dependency injection rather than hand-rolling this pattern — it keeps your classes unit-testable and avoids hidden global state. Be ready to discuss the enum-based Singleton as the most robust implementation.

02 Factory Method

Creational

Definition: Defines an interface for creating an object, but lets subclasses decide which concrete class to instantiate. The client only ever depends on the abstract product.

Real-world use case: A logistics app that needs to create Truck or Ship transport objects depending on delivery mode, without the ordering logic knowing the concrete class.

Spring Boot / real project: Spring's BeanFactory and ApplicationContext are themselves Factory Method implementations — getBean() hides which concrete class gets instantiated. You'll also write this pattern by hand for things like a NotificationSenderFactory that returns an Email, SMS, or Push sender based on a user's notification preference.

UML Diagram



How to read it: The factory's createNotification() is overridden per subclass; client code only ever talks to the Notification interface.

Java Example

JAVA

```java
interface Notification {
    void notify(String message);
}

class EmailNotification implements Notification {
    public void notify(String message) {
        System.out.println("Email: " + message);
    }
}

class SmsNotification implements Notification {
    public void notify(String message) {
        System.out.println("SMS: " + message);
    }
}

abstract class NotificationFactory {
    abstract Notification createNotification();

    public void send(String message) {
        Notification n = createNotification();
        n.notify(message);
    }
}

class SmsNotificationFactory extends NotificationFactory {
    Notification createNotification() {
        return new SmsNotification();
    }
}

// Usage
NotificationFactory factory = new SmsNotificationFactory();
```

WHEN TO USE

- A class can't anticipate which concrete class of object it needs to create ahead of time
- You want to localize the knowledge of which helper subclass to instantiate
- You're building a library/framework and want callers to extend which product gets created

WHEN NOT TO USE

- When there's only ever going to be one product type — plain new is clearer
- When you need to create whole families of related objects together (reach for Abstract Factory instead)
- When a simple static factory method or a Spring bean lookup already solves the problem

ADVANTAGES

- Removes the binding between the creator code and concrete product classes
- Single Responsibility: creation logic lives in one place, not scattered across the client
- Open/Closed: new product types can be added via new factory subclasses without touching existing code

DISADVANTAGES

- Requires a new subclass for every new product variant, which can multiply classes quickly
- Adds a layer of indirection that can feel like overkill for simple object creation

COMMON MISTAKES

- Using Factory Method when a plain constructor or a static factory method (of(), valueOf()) would be simpler
- Letting the factory leak concrete types back to the client through its return type instead of the abstract product
- Confusing it with Abstract Factory in interviews — Factory Method is one product, one method

DIFFERENCE: VS ABSTRACT FACTORY

Factory Method creates ONE product via one overridden method. Abstract Factory creates a whole FAMILY of related products (e.g. a UI kit with a matching Button + Checkbox for each theme) through multiple factory methods grouped on one interface.

IMPORTANT INTERVIEW QUESTIONS

Q. Is Factory Method the same as a simple static factory method like `List.of()`?
A. No — a static factory method is just a creational idiom (no inheritance involved). The GoF Factory Method pattern specifically relies on subclassing: a base class defers instantiation to an overridden method in a subclass.

Q. Where have you used this pattern in a real Spring Boot service?
A. A common example: a PaymentGatewayFactory that returns a Razorpay, Stripe, or PayU implementation of a PaymentGateway interface based on a merchant's configured provider, keeping the checkout service decoupled from any specific SDK.

SENIOR-LEVEL DISCUSSION

Factory Method creates ONE product via one overridden method — contrast this with Abstract Factory, which creates a whole family of related products. Interviewers often probe whether you reach for this pattern reflexively when a simple static factory method would do.

03 Abstract Factory

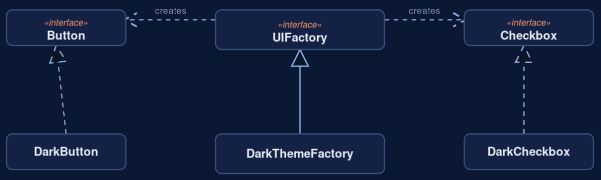
Creational

Definition: Produces families of related or dependent objects without specifying their concrete classes — one factory interface, multiple coordinated products.

Real-world use case: A cross-platform UI toolkit that must produce a matching Button + Checkbox for whichever theme (Light/Dark) is active, always as a consistent set.

Spring Boot / real project: Multi-tenant SaaS backends often use this: a TenantServiceFactory returns a coordinated bundle of tenant-specific beans (a custom PricingEngine + TaxCalculator + InvoiceFormatter) so every object created for a given tenant is guaranteed to be compatible with that tenant's rules.

UML Diagram



How to read it: One factory interface declares creation methods for every product in the family; each concrete factory (e.g. DarkThemeFactory) guarantees the products it returns match.

Java Example

JAVA

```java
interface Button { void render(); }
interface Checkbox { void render(); }

class DarkButton implements Button {
    public void render() { System.out.println("Rendering dark button"); }
}
class DarkCheckbox implements Checkbox {
    public void render() { System.out.println("Rendering dark checkbox"); }
}
class LightButton implements Button {
    public void render() { System.out.println("Rendering light button"); }
}
class LightCheckbox implements Checkbox {
    public void render() { System.out.println("Rendering light checkbox"); }
}

interface UIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

class DarkThemeFactory implements UIFactory {
    public Button createButton() { return new DarkButton(); }
    public Checkbox createCheckbox() { return new DarkCheckbox(); }
}

// Usage
UIFactory factory = new DarkThemeFactory();
factory.createButton().render();
factory.createCheckbox().render();
```

WHEN TO USE

- Your system needs to stay independent of how its products are created, composed, and represented
- You must enforce that a group of related products is always used together (never mix Dark button with Light checkbox)
- You want to swap an entire family of related objects at runtime or via configuration

WHEN NOT TO USE

- When there's only one product type to create (Factory Method is enough)
- When product families rarely change — the extra abstraction layer isn't earning its cost
- When Spring profiles/`@ConditionalOnProperty` beans already give you the swap-a-family behavior you need

ADVANTAGES

- Guarantees compatibility between products created by the same factory
- Isolates concrete classes from client code entirely
- Makes swapping an entire product family a one-line change (swap the factory)

DISADVANTAGES

- Adding a new product to the family means touching every concrete factory — violates Open/Closed at the family level
- Significant boilerplate for a small number of product variants

COMMON MISTAKES

- Reaching for Abstract Factory when only one product varies — that's just Factory Method
- Letting a concrete factory return a product from the wrong family (breaking the compatibility guarantee the pattern exists to protect)
- Over-engineering: building this pattern speculatively for 'future' product families that never materialize

DIFFERENCE: VS BUILDER

Abstract Factory produces different, independent products in one call each (a family). Builder constructs ONE complex object step-by-step. Don't confuse 'family of objects' with 'one object built in stages'.

IMPORTANT INTERVIEW QUESTIONS

Q. Why is Abstract Factory sometimes called 'a factory of factories'?
A. Because each concrete factory itself groups multiple factory methods (one per product type) behind a single interface — so instantiating a factory instantiates the ability to create an entire coordinated set of products.

Q. How do you extend an Abstract Factory to support a brand-new product family (e.g. a third 'HighContrast' theme)?
A. Add a new concrete factory (HighContrastThemeFactory) implementing UIFactory, plus matching HighContrastButton and HighContrastCheckbox classes. Existing client code that depends only on UIFactory doesn't change — but adding a new PRODUCT TYPE (e.g. a Slider) requires touching every factory.

SENIOR-LEVEL DISCUSSION

Abstract Factory is essentially a factory of factories — each concrete factory guarantees the products it returns are compatible with each other. Be ready to explain the trade-off: easy to add new families, hard to add new product types.

04 Builder

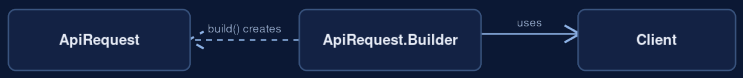
Creational

Definition: Separates the construction of a complex object from its representation, assembling it step by step through a fluent, readable API.

Real-world use case: Building an immutable HttpRequest or a multi-field ApiResponse object that has many optional parameters — avoids telescoping constructors.

Spring Boot / real project: UriComponentsBuilder, ResponseEntity.BodyBuilder, and Lombok's @Builder annotation are Builder pattern in daily use — Spring Boot engineers build request objects, response DTOs, and test fixtures with fluent builders constantly rather than writing constructors with 8+ parameters.

UML Diagram



How to read it: The Builder accumulates state through chained setter calls, then produces one immutable, fully-constructed product via build().

Java Example

JAVA

```java
public class ApiRequest {
    private final String url;
    private final String method;
    private final Map<String, String> headers;
    private final String body;

    private ApiRequest(Builder b) {
        this.url = b.url;
        this.method = b.method;
        this.headers = b.headers;
        this.body = b.body;
    }

    public static class Builder {
        private String url;
        private String method = "GET";
        private Map<String, String> headers = new HashMap<>();
        private String body;

        public Builder url(String url) { this.url = url; return this; }
        public Builder method(String method) { this.method = method; return this; }
        public Builder header(String k, String v) { headers.put(k, v); return this; }
        public Builder body(String body) { this.body = body; return this; }
        public ApiRequest build() { return new ApiRequest(this); }
    }
}

// Usage
ApiRequest req = new ApiRequest.Builder()
    .url("/api/orders")
    .method("POST")
    .header("Content-Type", "application/json")
    .body("{\"item\":\"book\"}")
```

WHEN TO USE

- The object has many optional parameters and a telescoping constructor would be unreadable
- You want the resulting object to be immutable once constructed
- Construction involves multiple steps that benefit from being named and readable at the call site

WHEN NOT TO USE

- The object has only 2-3 fields — a normal constructor is simpler and clearer
- You don't care about immutability and a plain setter-based POJO is acceptable
- Object creation is a single trivial step with no real 'assembly' involved

ADVANTAGES

- Reads far better at the call site than a long constructor argument list
- Enables step-by-step construction and validation, and easy support for optional fields
- Naturally produces immutable objects when the Builder itself is mutable but the product isn't

DISADVANTAGES

- Doubles the number of classes (product + builder), adding boilerplate for simple objects
- Without Lombok or similar codegen, hand-writing builders for many DTOs gets repetitive

COMMON MISTAKES

- Forgetting to validate required fields inside build(), letting an invalid object slip through
- Making the built product mutable anyway, defeating the main benefit of using Builder in the first place
- Reaching for a hand-written Builder when @Builder from Lombok (or a record with a compact constructor) already covers the need

DIFFERENCE: VS TELESCOPING CONSTRUCTORS

Prefer Builder over a constructor with 6+ parameters (the 'telescoping constructor' anti-pattern where you overload the constructor for every combination of optional fields) — Builder reads far better and scales to more optional fields without new overloads.

IMPORTANT INTERVIEW QUESTIONS

Q. How does Builder differ from a Director-based GoF Builder (with a separate Director class)?
A. The classic GoF version has a Director that knows the build sequence and calls Builder steps in order, useful when the same construction steps produce different representations. In modern Java, the fluent self-directed builder (client calls the steps itself, as shown here) is far more common and simpler.

Q. Would you use Builder or a Java record for an immutable DTO with 4 required fields?
A. A record — Builder earns its complexity when there are many OPTIONAL fields or multi-step validation. For a small number of required fields, a record's compact constructor is less code and equally immutable.

SENIOR-LEVEL DISCUSSION

Prefer Builder over a constructor with 6+ parameters (the 'telescoping constructor' anti-pattern) — it reads far better at the call site. In interviews, mention Lombok's @Builder as the pragmatic real-world shortcut over hand-writing this pattern.

05 Prototype

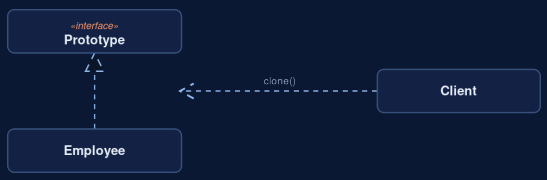
Creational

Definition: Creates new objects by cloning an existing instance ('the prototype') rather than instantiating from scratch — useful when creation is expensive.

Real-world use case: Duplicating a fully-configured Employee record or a complex game character object where re-running setup logic would be costly.

Spring Boot / real project: Spring itself supports prototype-scoped beans (@Scope("prototype")), which return a new instance on every lookup rather than reusing a singleton — handy for stateful, short-lived objects like a per-request builder or a mutable session-scoped helper.

UML Diagram



How to read it: The client asks an existing object to clone() itself instead of calling new and re-running expensive setup logic.

Java Example

JAVA

```
public class Employee implements Cloneable {
    private String name;
    private String department;
    private List<String> skills;

    public Employee(String name, String department, List<String> skills) {
        this.name = name;
        this.department = department;
        this.skills = new ArrayList<>(skills);
    }

    @Override
    public Employee clone() {
        // deep copy the mutable list to avoid shared references
        return new Employee(this.name, this.department, this.skills);
    }

    public void setName(String name) { this.name = name; }

    @Override
    public String toString() {
        return name + " - " + department + " - " + skills;
    }
}

// Usage
Employee original = new Employee("Riya", "Engineering", List.of("Java", "Spring"));
Employee copy = original.clone();
copy.setName("Riya (Clone)");
```

WHEN TO USE

- Creating an object from scratch is significantly more expensive than copying an existing one
- You need many near-identical objects that differ only in a couple of fields
- The concrete classes to instantiate are only known at runtime and you want to avoid a factory hierarchy

WHEN NOT TO USE

- Object graphs contain circular references or resources (sockets, file handles) that can't be meaningfully cloned
- Construction is cheap — cloning adds complexity for no real gain
- Deep-copy semantics would be ambiguous or error-prone for the object's fields

ADVANTAGES

- Avoids re-running potentially expensive initialization logic
- Lets you add/remove concrete product types at runtime by registering new prototypes, without a class hierarchy of factories
- Reduces subclassing compared to Factory Method in some designs

DISADVANTAGES

- Cloning objects with circular references or complex nested graphs is genuinely difficult to get right
- Java's built-in Cloneable is notoriously error-prone (shallow copy by default)

COMMON MISTAKES

- Relying on Object.clone()'s default shallow copy for objects containing mutable collections or nested objects
- Forgetting that Cloneable is a marker interface with no methods — implementing it without overriding clone() does nothing
- Not handling CloneNotSupportedException or exposing clone() as a checked-exception-throwing public API unnecessarily

DIFFERENCE: VS COPY CONSTRUCTOR

A copy constructor (`new Employee(original)`) achieves the same goal as Prototype's clone() with far fewer Java-specific gotchas — no CloneNotSupportedException, no shallow-copy trap, and it works naturally with final fields. Many senior engineers prefer it over implementing Cloneable.

IMPORTANT INTERVIEW QUESTIONS

Q. What's wrong with Java's Cloneable interface?
A. It's a marker interface with no clone() method of its own (clone() actually lives on Object as protected), the default implementation does a shallow field-by-field copy, and it doesn't play well with final fields or immutability. Effective Java recommends avoiding it in favor of copy constructors or static factory copy methods.

Q. When is Prototype clearly better than just calling `new` again?
A. When object construction involves expensive work you'd otherwise repeat — e.g., loading a large configuration from disk, running an expensive computation, or establishing a baseline network/DB state — and a copy with a couple of fields changed is far cheaper than doing that work again.

SENIOR-LEVEL DISCUSSION

Java's Cloneable is notoriously tricky (shallow vs deep copy pitfalls) — in interviews, mention implementing a copy constructor as a safer, more idiomatic alternative to Object.clone().

01 Adapter

Structural

Definition: Converts the interface of one class into another interface that a client expects, allowing otherwise-incompatible classes to work together.

Real-world use case: Wrapping a legacy XmlPaymentGateway behind a modern PaymentProcessor interface so new code never has to know XML exists.

Spring Boot / real project: Wrapping a third-party SDK client (say, an old SOAP-based vendor library) behind your own domain interface so the rest of the Spring app depends only on your abstraction — this is the Adapter pattern, and it's exactly what a hexagonal-architecture 'port and adapter' does at the infrastructure boundary.

UML Diagram

How to read it: The Adapter implements the interface clients expect, while internally delegating each call to the incompatible legacy object it wraps.



Java Example

JAVA

```java
class LegacyXmlPaymentGateway {
    public String payInXml(String xmlPayload) {
        return "XML payment processed: " + xmlPayload;
    }
}

interface PaymentProcessor {
    void pay(double amount);
}

class XmlPaymentAdapter implements PaymentProcessor {
    private final LegacyXmlPaymentGateway legacyGateway;

    public XmlPaymentAdapter(LegacyXmlPaymentGateway legacyGateway) {
        this.legacyGateway = legacyGateway;
    }

    public void pay(double amount) {
        String xml = "<payment><amount>" + amount + "</amount></payment>";
        System.out.println(legacyGateway.payInXml(xml));
    }
}

// Usage
PaymentProcessor processor = new XmlPaymentAdapter(new LegacyXmlPaymentGateway());
processor.pay(499.00);
```

WHEN TO USE

- You want to reuse an existing class but its interface doesn't match what the rest of your system expects
- You're integrating a third-party library or legacy code you cannot modify
- You want to create a reusable class that cooperates with classes that don't share a compatible interface

WHEN NOT TO USE

- You control both interfaces — just change one of them instead of adding a translation layer
- The mismatch is trivial (a rename) — sometimes a simple wrapper method is enough without a formal pattern

ADVANTAGES

- Lets incompatible interfaces work together without modifying either existing class
- Follows Single Responsibility — the translation logic lives in one dedicated class
- Makes legacy or third-party code swappable behind your own stable interface

DISADVANTAGES

- Adds an extra layer of indirection and another class to maintain
- Overuse across a codebase can obscure how many real translation layers exist between components

COMMON MISTAKES

- Adapting an interface so heavily that it silently changes behavior instead of just translating shape — that's scope creep beyond what Adapter should do
- Confusing Adapter (interface translation) with Decorator (behavior addition), even though both wrap an object
- Writing a two-way adapter that implements both interfaces bidirectionally when a simple one-way translation was all that was needed

DIFFERENCE: VS FACADE

Adapter makes ONE existing interface compatible with what a client expects (translation of a single class). Facade creates a brand-new simplified interface over a WHOLE subsystem of multiple classes (simplification, not translation).

IMPORTANT INTERVIEW QUESTIONS

Q. Class adapter vs object adapter — what's the difference?
A. A class adapter uses multiple inheritance (not available for classes in Java) to inherit both the target interface and the adaptee. An object adapter — the standard Java approach — holds a reference to the adaptee and delegates calls, which is more flexible since Java only allows single class inheritance.

Q. How is Adapter different from just adding a new method to the existing class?
A. You often can't modify the existing/legacy/third-party class at all (closed for modification, no source access). Adapter solves this without touching the original class, honoring the Open/Closed Principle.

SENIOR-LEVEL DISCUSSION

Adapter changes an interface to make two things compatible after the fact — it doesn't add new behavior, it translates existing behavior. Interviewers like to hear you distinguish this cleanly from Decorator and Facade.

02 Bridge

Structural

Definition: Decouples an abstraction from its implementation so the two class hierarchies can vary and evolve independently of each other.

Real-world use case: A messaging feature (Urgent / Normal) that must work across multiple delivery channels (Email / SMS) without an explosion of subclasses per combination.

Spring Boot / real project: A reporting service that must support several report TYPES (Sales, Inventory) across several output FORMATS (PDF, Excel, CSV) is a textbook Bridge candidate in a Spring service layer — each ReportGenerator abstraction holds a reference to an Exporter implementor injected as a bean, so adding a new format never touches report logic.

UML Diagram



How to read it: Message (abstraction) holds a reference to a MessageSender (implementor) instead of inheriting from it — the two hierarchies vary independently.

Java Example

JAVA

```java
interface MessageSender {
    void sendMessage(String text);
}

class EmailSender implements MessageSender {
    public void sendMessage(String text) { System.out.println("Email: " + text); }
}
class SmsSender implements MessageSender {
    public void sendMessage(String text) { System.out.println("SMS: " + text); }
}

abstract class Message {
    protected MessageSender sender;
    Message(MessageSender sender) { this.sender = sender; }
    abstract void send(String text);
}

class UrgentMessage extends Message {
    UrgentMessage(MessageSender sender) { super(sender); }
    void send(String text) { sender.sendMessage("URGENT: " + text); }
}

// Usage
Message msg = new UrgentMessage(new SmsSender());
msg.send("Server is down");
```

WHEN TO USE

- You have two dimensions of variation (e.g. message urgency × delivery channel) that would otherwise multiply into N x M subclasses
- You want to switch implementations at runtime, not just compile-time
- You need to share an implementation across several abstraction hierarchies

WHEN NOT TO USE

- Only one dimension of variation actually exists — a normal inheritance hierarchy is simpler
- The abstraction and implementation are tightly coupled by design and will never vary independently

ADVANTAGES

- Avoids permanent binding between an abstraction and one implementation — both can extend independently
- Eliminates the N x M subclass explosion of naive multiple inheritance
- Implementation details are fully hidden from client code

DISADVANTAGES

- Increases the number of interfaces/classes and up-front design complexity
- Can be harder to grasp than a straightforward inheritance hierarchy for a simple case

COMMON MISTAKES

- Confusing Bridge with Adapter — Bridge is designed upfront to let two hierarchies vary independently; Adapter is retrofitted after the fact to fix an incompatibility
- Adding a Bridge for a single dimension of variation where simple inheritance or Strategy already solves it

DIFFERENCE: VS ADAPTER

Bridge is Composition over Inheritance in its purest form, designed UP FRONT to let an abstraction and its implementation vary independently — it avoids the N x M class explosion you'd get subclassing every combination. Adapter, by contrast, is applied AFTER THE FACT to make two already-existing incompatible interfaces work together.

IMPORTANT INTERVIEW QUESTIONS

Q. How is Bridge different from Strategy?
A. Structurally they look similar (composition + an interface), but intent differs: Bridge is a structural pattern splitting an ABSTRACTION hierarchy from an IMPLEMENTATION hierarchy so both can grow. Strategy is a behavioral pattern for swapping one ALGORITHM at runtime — there's usually no separate 'abstraction hierarchy' on the client side.

Q. Give a concrete example of the N x M explosion Bridge avoids.
A. Without Bridge, supporting {Urgent, Normal} messages across {Email, SMS, Push} channels via inheritance alone would need up to 6 subclasses (UrgentEmailMessage, NormalEmailMessage, UrgentSmsMessage...). With Bridge, you need just 2 Message subclasses and 3 MessageSender implementations, combined freely at runtime.

SENIOR-LEVEL DISCUSSION

Bridge is Composition over Inheritance in its purest form — it avoids the N x M class explosion you'd get subclassing every combination. Senior interviewers often ask you to sketch the class explosion first, then show how Bridge collapses it.

03 Composite

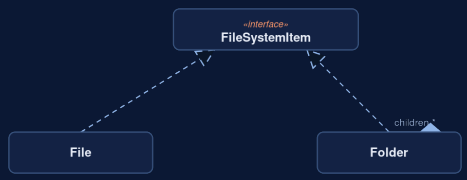
Structural

Definition: Composes objects into tree structures to represent part-whole hierarchies, letting clients treat individual objects and compositions uniformly.

Real-world use case: A file-system explorer where a Folder can contain Files and other Folders, yet both respond to the same getSize() call.

Spring Boot / real project: Permission/menu trees in an admin panel — a MenuItem that can either be a leaf action or a Menu containing child MenuItems — are commonly modeled with Composite, letting the rendering and authorization code call the same `isVisibleTo(user)` method regardless of tree depth.

UML Diagram



How to read it: Folder both implements FileSystemItem and holds a collection of FileSystemItem — that self-reference is what lets trees nest to any depth.

Java Example

JAVA

```java
interface FileSystemItem {
    int getSize();
}

class File implements FileSystemItem {
    private int size;
    File(int size) { this.size = size; }
    public int getSize() { return size; }
}

class Folder implements FileSystemItem {
    private List<FileSystemItem> items = new ArrayList<>();
    public void add(FileSystemItem item) { items.add(item); }
    public int getSize() {
        int total = 0;
        for (FileSystemItem item : items) {
            total += item.getSize(); // recurses into sub-folders
        }
        return total;
    }
}

// Usage
Folder root = new Folder();
root.add(new File(120));
Folder sub = new Folder();
sub.add(new File(80));
root.add(sub);
System.out.println("Total size: " + root.getSize()); // 200
```

WHEN TO USE

- You need to represent part-whole hierarchies (trees) of objects
- Clients should be able to ignore the difference between individual objects and compositions of objects
- The tree structure can be arbitrarily deep and recursive operations (sum, render, validate) need to work uniformly

WHEN NOT TO USE

- Your data is genuinely flat with no nesting — the extra interface adds needless indirection
- Leaf and composite nodes need such different behavior that a uniform interface would need lots of no-op or exception-throwing methods

ADVANTAGES

- Client code treats leaves and composites identically via polymorphism — no type-checking if/else
- Makes it easy to add new kinds of components without breaking existing client code
- Naturally models recursive, tree-shaped domain concepts

DISADVANTAGES

- Can make the design overly general — it's tempting to put operations on the interface that don't make sense for every component
- Runtime type-checking is sometimes still needed if leaf-only or composite-only operations exist

COMMON MISTAKES

- Putting child-management methods (add/remove) on the Component interface itself, forcing leaves to implement no-op or exception-throwing versions
- Forgetting to guard against cycles when building the tree dynamically (a Folder accidentally containing itself)

DIFFERENCE: VS DECORATOR

Both Composite and Decorator recursively wrap objects behind a common interface, but their INTENT differs: Composite models a whole PART-WHOLE tree (many children under one parent). Decorator wraps exactly ONE object to add extra behavior — it's a chain, not a tree.

IMPORTANT INTERVIEW QUESTIONS

Q. Should add()/remove() live on the Component interface or only on Composite?
A. The GoF book puts them on Component for transparency (uniform treatment) at the cost of leaves having to implement meaningless methods. Most modern Java code instead puts add()/remove() only on the Composite subclass for safety, accepting a bit less uniformity — a trade-off worth discussing explicitly in interviews.

Q. How would you compute the total size of a folder tree without recursion overflowing on very deep trees?
A. Switch the recursive getSize() to an iterative traversal using an explicit Deque/Stack of FileSystemItem to simulate the call stack, avoiding StackOverflowError on pathologically deep hierarchies.

SENIOR-LEVEL DISCUSSION

The key win: client code calling component.operation() never needs an if/else to check 'is this a leaf or a group?' — polymorphism handles it. File systems, UI component trees, and org-chart hierarchies are the classic examples to cite.

04 Decorator

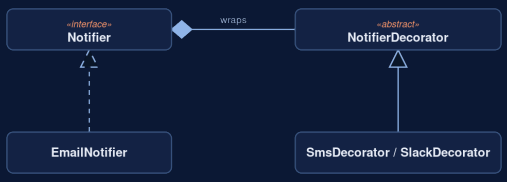
Structural

Definition: Attaches new responsibilities to an object dynamically at runtime by wrapping it, as a flexible alternative to subclassing.

Real-world use case: Layering extra alert channels (SMS, Slack) on top of a base Notifier without touching existing classes — common in microservice alerting.

Spring Boot / real project: Spring's own HttpServletRequestWrapper and the whole Servlet Filter chain lean on Decorator, and you'll write it directly when layering cross-cutting concerns like caching, retry, or rate-limiting around a service interface (e.g. `new RetryingPaymentService(new CachingPaymentService(new PaymentServiceImpl()))`).

UML Diagram



How to read it: NotifierDecorator implements the same interface it wraps and forwards calls to the wrapped object — decorators can be chained to any depth.

Java Example

JAVA

```java
interface Notifier {
    void send(String message);
}

class EmailNotifier implements Notifier {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

abstract class NotifierDecorator implements Notifier {
    protected Notifier wrapped;
    NotifierDecorator(Notifier wrapped) { this.wrapped = wrapped; }
}

class SmsDecorator extends NotifierDecorator {
    SmsDecorator(Notifier wrapped) { super(wrapped); }
    public void send(String message) {
        wrapped.send(message);
        System.out.println("SMS: " + message);
    }
}

class SlackDecorator extends NotifierDecorator {
    SlackDecorator(Notifier wrapped) { super(wrapped); }
    public void send(String message) {
        wrapped.send(message);
        System.out.println("Slack: " + message);
    }
}

// Usage
Notifier notifier = new SlackDecorator(new SmsDecorator(new EmailNotifier()));
```

WHEN TO USE

- You need to add responsibilities to individual objects dynamically and transparently, without affecting other objects
- Extension by subclassing would be impractical due to an explosion of combinations
- You want responsibilities that can be withdrawn as well as added

WHEN NOT TO USE

- There's only ever one fixed combination of behaviors — a single class with the logic inline is simpler
- You need to control ACCESS to the wrapped object rather than add behavior (use Proxy)

ADVANTAGES

- Adds behavior at runtime, more flexible than static inheritance
- Avoids class explosion — you don't need an EmailSmsSlackNotifier for every combination
- Each decorator follows Single Responsibility, handling exactly one added concern

DISADVANTAGES

- Many small wrapper objects can make debugging and stack traces harder to follow
- Order of wrapping can matter and is easy to get wrong silently

COMMON MISTAKES

- Forgetting to call the wrapped object's method before/after adding new behavior, silently dropping the base behavior
- Using Decorator when the real intent is access control (should be Proxy) rather than adding responsibilities
- Letting decorators depend on concrete decorator types instead of only the common interface, breaking composability

DIFFERENCE: VS PROXY

Decorator vs Inheritance: Decorator adds behavior at runtime and avoids class explosion. Decorator vs Proxy: both wrap an object behind the same interface, but Decorator ADDS new responsibilities/behavior, while Proxy CONTROLS ACCESS to the object (lazy init, security, caching) without changing its core behavior.

IMPORTANT INTERVIEW QUESTIONS

Q. How does Java I/O (`new BufferedReader(new FileReader(...))`) relate to this pattern?
A. It's the textbook real-world example of Decorator — Reader/InputStream wrapper classes each add one capability (buffering, line-reading) around a base stream, and can be layered in any combination.

Q. Can Decorator and Proxy be combined?
A. Yes — it's common to wrap a real service first in a Proxy (for lazy loading or access control) and then in one or more Decorators (for logging, retry, caching), since both implement the same target interface and are fully composable.

SENIOR-LEVEL DISCUSSION

Decorator vs Inheritance: it adds behavior at runtime and avoids class explosion (you don't need an EmailSmsSlackNotifier for every combination). Be ready to contrast it precisely against Proxy in the same breath.

05 Facade

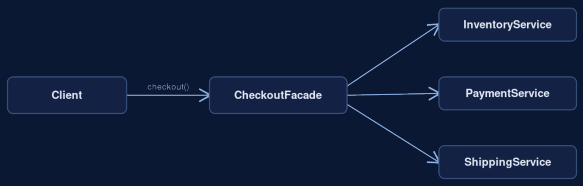
Structural

Definition: Provides a simplified, unified interface to a larger and more complex subsystem, hiding its internal wiring from the client.

Real-world use case: A checkout() method that internally coordinates InventoryService, PaymentService, and ShippingService behind one simple call.

Spring Boot / real project: A Spring @Service layer sitting in front of several lower-level repositories, external API clients, and validators is Facade in its most common Spring Boot form — controllers call one clean `OrderFacadeService.placeOrder()` method instead of orchestrating five collaborators themselves.

UML Diagram



How to read it: The Facade hides three separate subsystem calls behind one method; the client never touches Inventory, Payment, or Shipping services directly.

Java Example

JAVA

```java
class InventoryService {
    boolean reserve(String item) {
        System.out.println("Reserved: " + item);
        return true;
    }
}
class PaymentService {
    boolean charge(double amount) {
        System.out.println("Charged: " + amount);
        return true;
    }
}
class ShippingService {
    void schedule(String item) {
        System.out.println("Shipping scheduled: " + item);
    }
}

// Facade
class CheckoutFacade {
    private InventoryService inventory = new InventoryService();
    private PaymentService payment = new PaymentService();
    private ShippingService shipping = new ShippingService();

    public void checkout(String item, double amount) {
        if (inventory.reserve(item) && payment.charge(amount)) {
            shipping.schedule(item);
        }
    }
}

// Usage
new CheckoutFacade().checkout("Java Interview Kit", 999.00);
```

WHEN TO USE

- A subsystem is complex and most clients only need a simple, common-path interaction with it
- You want to decouple client code from a subsystem's internals to reduce coupling and ease future changes
- You're layering a system and want a clean entry point per layer

WHEN NOT TO USE

- Clients genuinely need fine-grained access to individual subsystem parts — an overly restrictive facade forces workarounds
- The 'subsystem' is really just one class already — there's nothing to simplify

ADVANTAGES

- Shields client code from subsystem complexity and internal changes
- Reduces coupling between client code and the number of classes it must know about
- Provides a natural place to enforce an ordering or business rule across subsystem calls

DISADVANTAGES

- Can become a 'god object' if it accumulates every use case instead of staying a thin coordination layer
- Doesn't prevent access to the subsystem directly, so discipline is needed to keep the facade as the primary entry point

COMMON MISTAKES

- Letting the Facade grow business logic that really belongs in the subsystem services it coordinates
- Assuming Facade restricts access — it doesn't; nothing stops a client from bypassing it and calling subsystem classes directly unless you deliberately hide them

DIFFERENCE: VS ADAPTER

Adapter makes ONE existing interface compatible with what a client expects (translation). Facade creates a brand-new SIMPLIFIED interface over a whole subsystem of MULTIPLE classes (simplification). Facade doesn't stop you from using the subsystem directly when you need to — it just gives most callers a much simpler default entry point.

IMPORTANT INTERVIEW QUESTIONS

Q. Does a Facade have to hide the subsystem completely?
A. No — Facade doesn't stop you from using the subsystem directly when you need finer control; it just gives most callers a much simpler default entry point. Advanced clients can still reach the underlying InventoryService, PaymentService, etc. directly.

Q. How would you keep a Facade from becoming a 'god class' over time?
A. Keep it thin — it should only orchestrate calls and sequencing, not contain business rules. If it starts accumulating unrelated use cases, split it into multiple, more focused facades per bounded context or workflow.

SENIOR-LEVEL DISCUSSION

Facade doesn't stop you from using the subsystem directly when you need to — it just gives most callers a much simpler default entry point. This is the most-asked distinction versus Adapter, so rehearse it precisely.

06 Flyweight

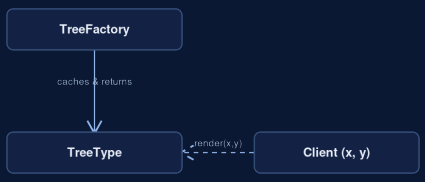
Structural

Definition: Minimizes memory usage by sharing as much data as possible with other similar objects — separates intrinsic (shared) from extrinsic (per-instance) state.

Real-world use case: Rendering millions of trees in a game world where the mesh/texture (intrinsic) is shared but x/y position (extrinsic) differs per tree.

Spring Boot / real project: Java's own Integer.valueOf() cache (-128 to 127) and String interning are Flyweight in the standard library. In backend services, Flyweight shows up when caching immutable, expensive-to-build lookup objects — e.g. a shared `TaxRuleSet` per country code reused across millions of order calculations instead of rebuilt per request.

UML Diagram



How to read it: TreeFactory caches shared TreeType flyweights by their intrinsic key; extrinsic data (x, y) is supplied by the caller on every render() call instead of stored.

Java Example

JAVA

```
class TreeType { // intrinsic (shared) state
    private final String mesh;
    private final String texture;
    TreeType(String mesh, String texture) {
        this.mesh = mesh;
        this.texture = texture;
    }
    void render(int x, int y) { // extrinsic state passed in
        System.out.println("Rendering " + mesh + " at (" + x + "," + y + ")");
    }
}

class TreeFactory {
    private static Map<String, TreeType> cache = new HashMap<>();
    static TreeType getTreeType(String mesh, String texture) {
        String key = mesh + texture;
        return cache.computeIfAbsent(key, k -> new TreeType(mesh, texture));
    }
}

// Usage — thousands of trees, only a few TreeType objects ever created
TreeType oak = TreeFactory.getTreeType("oak.mesh", "bark.png");
oak.render(10, 20);
oak.render(15, 45);
```

WHEN TO USE

- An application uses a huge number of objects that share most of their state
- Object storage cost is a real, measured problem (large-scale simulations, rendering, big in-memory caches)
- State can be cleanly split into intrinsic (shareable) and extrinsic (context-dependent) parts

WHEN NOT TO USE

- The number of objects is small enough that memory isn't actually a concern — the added complexity isn't worth it
- State can't be cleanly separated into intrinsic vs extrinsic without awkward workarounds

ADVANTAGES

- Dramatically reduces memory footprint when many objects share common data
- Centralizes and caches expensive shared state in one factory

DISADVANTAGES

- Adds real design complexity — the extrinsic/intrinsic split has to be threaded through every call site
- Shared mutable state is dangerous — flyweights must be treated as immutable or carefully synchronized

COMMON MISTAKES

- Accidentally storing extrinsic (per-instance) data inside the shared flyweight object, reintroducing the memory problem and creating subtle shared-state bugs
- Applying Flyweight prematurely without profiling — it's a targeted optimization, not a default design choice

DIFFERENCE: VS SINGLETON

Singleton guarantees exactly ONE instance of a class exists. Flyweight allows MANY shared instances, keyed by their intrinsic state, so that logically-identical objects are reused instead of duplicated — the cache can hold any number of distinct flyweights.

IMPORTANT INTERVIEW QUESTIONS

Q. What's the classic real-world example of Flyweight?
A. Java's Integer.valueOf() cache for values -128 to 127, and String literal interning in the string pool — both reuse a shared, immutable instance instead of allocating a new object for every occurrence of the same value.

Q. The classic exam question: what's shared vs passed in?
A. Intrinsic state (shared, stored once inside the flyweight — e.g. the tree's mesh/texture) vs extrinsic state (supplied by the client on every call, never stored in the flyweight — e.g. the tree's x/y position).

SENIOR-LEVEL DISCUSSION

The classic exam question: what's shared (intrinsic, stored once in the flyweight) vs what's passed in per call (extrinsic, supplied by the client)? Getting this distinction crisp is the single biggest signal in a Flyweight discussion.

07 Proxy

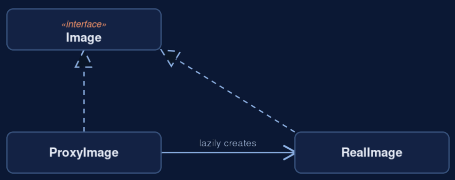
Structural

Definition: Provides a placeholder or surrogate for another object to control access to it — for lazy loading, security checks, logging, or caching.

Real-world use case: A ProfileImageProxy that defers loading a large image from disk until display() is actually called for the first time.

Spring Boot / real project: Spring AOP is built entirely on dynamic proxies (JDK interface proxies or CGLIB class proxies) — every time you use @Transactional, @Cacheable, or @Async, Spring wraps your bean in a runtime-generated Proxy that adds that cross-cutting behavior before delegating to your real method.

UML Diagram



How to read it: ProxyImage implements the same Image interface as RealImage and controls exactly when (or whether) the real, expensive object gets created.

Java Example

JAVA

```java
interface Image {
    void display();
}

class RealImage implements Image {
    private String filename;
    RealImage(String filename) {
        this.filename = filename;
        loadFromDisk();
    }
    private void loadFromDisk() {
        System.out.println("Loading " + filename + " from disk...");
    }
    public void display() {
        System.out.println("Displaying " + filename);
    }
}

class ProxyImage implements Image {
    private String filename;
    private RealImage realImage; // not created until needed
    ProxyImage(String filename) { this.filename = filename; }
    public void display() {
        if (realImage == null) {
            realImage = new RealImage(filename); // lazy init
        }
        realImage.display();
    }
}

// Usage
Image image = new ProxyImage("profile.png");
image.display(); // loads + displays only now
```

WHEN TO USE

• You need lazy initialization of an expensive object that might not always be used
• You want to add access control, logging, caching, or remote-call handling transparently
• You want cross-cutting concerns applied without modifying the real object's code (Spring AOP style)

WHEN NOT TO USE

• The wrapped object is cheap to create and access control/lazy loading add no value
• You actually want to ADD new behavior/responsibilities rather than control access — that's Decorator

ADVANTAGES

• Controls access to the real object without the client needing to know a proxy is involved
• Enables lazy loading, caching, and security checks transparently
• Is exactly how Spring AOP implements @Transactional, @Cacheable, and @Async under the hood

DISADVANTAGES

• Adds an indirection layer that can complicate debugging (e.g. self-invocation issues with Spring AOP proxies)
• Response time can increase slightly due to the extra hop through the proxy

COMMON MISTAKES

• Calling a proxied method from WITHIN the same Spring bean (self-invocation) and being surprised that @Transactional/@Cacheable doesn't apply — the call bypasses the proxy entirely
• Confusing Proxy's intent (access control) with Decorator's intent (behavior addition) even though the code often looks nearly identical

DIFFERENCE: VS DECORATOR

Decorator adds new behavior; Proxy controls access to the same behavior. Spring AOP's @Transactional is a real-world proxy in action — it decides WHEN and WHETHER to let a call through to the real object (wrapping it in a transaction), not what new behavior the object itself has.

IMPORTANT INTERVIEW QUESTIONS

Q. Why does @Transactional silently not work on a method called from another method in the same class?
A. Spring's @Transactional relies on a proxy wrapping the bean; self-invocation (this.method()) bypasses the proxy entirely and calls the real object directly, skipping the transactional advice — a very common production bug rooted directly in how the Proxy pattern is implemented.

Q. JDK dynamic proxy vs CGLIB proxy — when does Spring use which?
A. Spring uses a JDK dynamic proxy (interface-based) when the bean implements at least one interface, and falls back to a CGLIB proxy (subclassing the concrete class) when it doesn't — both are runtime implementations of the Proxy pattern.

SENIOR-LEVEL DISCUSSION

Decorator adds new behavior; Proxy controls access to the same behavior. Spring AOP's @Transactional is a real-world proxy in action — this is one of the most commonly asked 'explain a pattern you've hit in production' interview questions.

01 Strategy

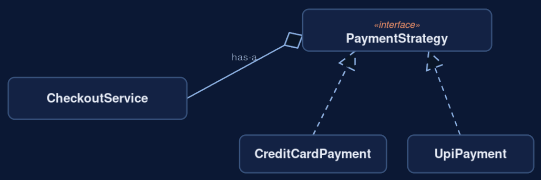
Behavioral

Definition: Defines a family of interchangeable algorithms, encapsulates each one, and lets the client swap between them at runtime.

Real-world use case: Swapping payment algorithms (Card, UPI, Wallet) at runtime without touching checkout logic — the shape used for Spring's pluggable auth.

Spring Boot / real project: Injecting a `List` and picking the right one at runtime by a discriminator (or letting Spring autowire a specific qualified bean) is a very common Spring Boot idiom — Spring Security's `AuthenticationProvider` chain and password encoders are built the same way.

UML Diagram



How to read it: CheckoutService holds a reference to the PaymentStrategy interface; the concrete strategy is injected/swapped from outside, never chosen internally.

Java Example

JAVA

```
interface PaymentStrategy {
    void pay(double amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid Rs. " + amount + " via Credit Card");
    }
}
class UpiPayment implements PaymentStrategy {
    public void pay(double amount) {
        System.out.println("Paid Rs. " + amount + " via UPI");
    }
}

class CheckoutService {
    private PaymentStrategy strategy;
    public CheckoutService(PaymentStrategy strategy) { this.strategy = strategy; }
    public void setStrategy(PaymentStrategy strategy) { this.strategy = strategy; }
    public void checkout(double amount) { strategy.pay(amount); }
}

// Usage
CheckoutService checkout = new CheckoutService(new UpiPayment());
checkout.checkout(1499.00);
checkout.setStrategy(new CreditCardPayment());
checkout.checkout(2999.00);
```

WHEN TO USE

- You have several variants of an algorithm and want to switch between them at runtime
- You want to avoid a large conditional (if/switch) that picks behavior based on type
- You need to isolate algorithm-specific logic from the code that uses it, for testability

WHEN NOT TO USE

- There's genuinely only one algorithm and it's unlikely to ever vary — the extra interface is unnecessary ceremony
- The 'strategies' don't actually share a meaningful common interface, forcing an awkward abstraction

ADVANTAGES

- Eliminates conditional logic for selecting behavior, replacing it with polymorphism
- New strategies can be added without touching existing code (Open/Closed)
- Each algorithm is independently testable in isolation

DISADVANTAGES

- Clients must be aware of the different strategies to choose the right one
- Increases the number of objects/classes in the system

COMMON MISTAKES

- Reinventing Strategy with a big if/else or switch statement instead of actually using polymorphism
- Confusing it with State, and not realizing WHO decides the transition (the client, for Strategy) is the key differentiator

DIFFERENCE: VS STATE

Strategy vs State — structurally almost identical (both hold a reference to an interface and delegate), but intent differs: Strategy is CHOSEN BY THE CLIENT from outside and doesn't change itself. State TRANSITIONS ITSELF internally as the object's own logic reacts to events.

IMPORTANT INTERVIEW QUESTIONS

Q. How would you avoid the client needing to know every concrete Strategy class?
A. Use a StrategyFactory or a Spring-managed map of beans (e.g. Map autowired by bean name) keyed by a discriminator, so client code just asks for 'upi' or 'card' rather than instantiating concrete classes directly.

Q. Is Strategy overkill for something as simple as a Comparator?
A. No — java.util.Comparator is itself a real-world Strategy: Collections.sort(list, comparator) lets you swap the sort algorithm/criteria at runtime without changing the sorting code itself.

SENIOR-LEVEL DISCUSSION

Strategy vs State — structurally almost identical, but Strategy is chosen by the client from outside; State transitions itself internally. This distinction is one of the most frequently asked in senior Java interviews.

02 Observer

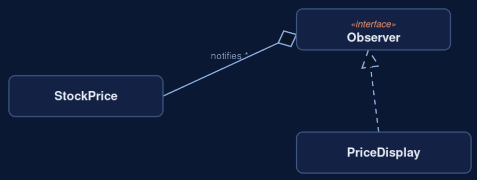
Behavioral

Definition: Defines a one-to-many dependency so that when one object (the subject) changes state, all its dependents (observers) are notified automatically.

Real-world use case: A StockPrice subject that pushes updates to multiple UI widgets and alerting services the instant the price changes.

Spring Boot / real project: Spring's built-in ApplicationEventPublisher and @EventListener are a production-grade Observer implementation — publishing an `OrderPlacedEvent` lets multiple decoupled listeners (email service, inventory service, analytics) react without the order service knowing any of them exist.

UML Diagram



How to read it: StockPrice (subject) keeps a list of Observer references and calls update() on each whenever its own state changes — subject and observers stay loosely coupled.

Java Example

JAVA

```java
interface Observer {
    void update(double price);
}

class StockPrice {
    private List<Observer> observers = new ArrayList<>();
    private double price;
    public void subscribe(Observer o) { observers.add(o); }
    public void setPrice(double price) {
        this.price = price;
        notifyObservers();
    }
    private void notifyObservers() {
        for (Observer o : observers) {
            o.update(price);
        }
    }
}

class PriceDisplay implements Observer {
    public void update(double price) {
        System.out.println("Display updated: Rs. " + price);
    }
}

// Usage
StockPrice stock = new StockPrice();
stock.subscribe(new PriceDisplay());
stock.setPrice(2450.75);
```

WHEN TO USE

- A change to one object must trigger updates to an open-ended, dynamic set of other objects
- You want to broadcast state changes without the subject knowing concrete observer classes
- You're building event-driven or pub/sub-style architecture within a single process

WHEN NOT TO USE

- There's exactly one, fixed dependent — a direct method call is simpler than the machinery of subscribe/notify
- Notification order and consistency matter a lot and a synchronous fan-out could cause subtle bugs (consider a message broker instead)

ADVANTAGES

- Subject and observers are loosely coupled — the subject only knows the Observer interface
- Supports broadcast communication and dynamic add/remove of listeners at runtime
- Matches Spring's own eventing model, so it composes cleanly with the framework

DISADVANTAGES

- Notification order isn't guaranteed by the pattern itself and can cause subtle bugs if observers depend on ordering
- Memory leaks are common if observers subscribe but are never unsubscribed (classic 'lapsed listener' problem)

COMMON MISTAKES

- Forgetting to unsubscribe observers, causing memory leaks — especially in long-lived subjects with short-lived observers
- Letting an observer throw an exception that stops the notification loop from reaching the remaining observers
- Doing heavy, blocking work inside update() on a synchronous notify, stalling the whole notification chain

DIFFERENCE: VS MEDIATOR

Observer is one-to-MANY: a subject broadcasts to any number of independent observers who don't know about each other. Mediator is many-to-many coordination THROUGH a hub: colleagues talk to each other via one central object rather than one object broadcasting outward.

IMPORTANT INTERVIEW QUESTIONS

Q. How does Spring's @EventListener relate to this pattern?
A. ApplicationEventPublisher.publishEvent() is the Subject's notify step, and every @EventListener method is an Observer's update() — Spring decouples them further by matching listeners to events by type rather than requiring an explicit subscribe() call.

Q. What's the 'lapsed listener' problem and how do you avoid it?
A. It's a memory leak where observers are subscribed but never unsubscribed, so the subject holds strong references to observers that should have been garbage collected. Mitigate it with explicit unsubscribe() calls, try-with-resources-style listener registrations, or WeakReference-based observer lists.

SENIOR-LEVEL DISCUSSION

This is the pattern behind pub/sub systems, Kafka consumers, and Java's own PropertyChangeListener — extremely common interview follow-up territory, especially probing the lapsed-listener memory leak.

03 Command

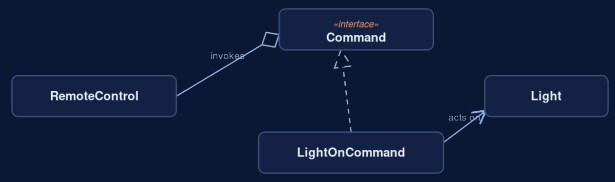
Behavioral

Definition: Encapsulates a request (and everything needed to execute it) as an object, enabling queuing, logging, undo/redo, and decoupled invokers.

Real-world use case: A text editor's undo stack, or a job queue where each task is a Command object that can be scheduled, retried, or logged independently.

Spring Boot / real project: Background job processing (a `Runnable`/`Callable` submitted to a Spring `TaskExecutor`, or a message payload deserialized into a Command object and dispatched to a handler) is Command in production — each unit of work is a self-contained object that can be queued, retried, and logged independently of who triggered it.

UML Diagram



How to read it: The invoker (RemoteControl) only ever calls execute() on the Command interface — it never knows which receiver (Light) or action is behind it.

Java Example

JAVA

```
interface Command {
    void execute();
}

class Light {
    void on() { System.out.println("Light ON"); }
    void off() { System.out.println("Light OFF"); }
}

class LightOnCommand implements Command {
    private Light light;
    LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.on(); }
}
class LightOffCommand implements Command {
    private Light light;
    LightOffCommand(Light light) { this.light = light; }
    public void execute() { light.off(); }
}

class RemoteControl {
    private List<Command> history = new ArrayList<>();
    public void press(Command command) {
        command.execute();
        history.add(command);
    }
}

// Usage
Light light = new Light();
RemoteControl remote = new RemoteControl();
remote.press(new LightOnCommand(light));
remote.press(new LightOffCommand(light));
```

WHEN TO USE

• You need to parameterize objects with an action to perform (queue it, log it, schedule it, undo it)
• You want to decouple the object that invokes an operation from the one that knows how to perform it
• You need undo/redo, transactional behavior, or a history of executed operations

WHEN NOT TO USE

• The action is simple and immediate — wrapping a one-line method call in a full Command object is overkill
• You don't need queuing, undo, logging, or decoupled invocation — a direct method call is clearer

ADVANTAGES

• Decouples the invoker from the receiver completely
• Naturally supports undo/redo, logging, and queuing of requests
• New commands can be added without changing invoker code (Open/Closed)

DISADVANTAGES

• Can lead to a large number of small command classes for simple actions
• Undo logic (if needed) adds real complexity — commands must capture enough state to reverse themselves

COMMON MISTAKES

• Building a Command object for every trivial action, ballooning the class count for no real benefit
• Forgetting to store enough state in the command to actually implement undo() when that requirement surfaces later

DIFFERENCE: VS STRATEGY

Both wrap behavior behind an interface, but Command represents a REQUEST/ACTION to be invoked later (often queued, logged, or undone), while Strategy represents an interchangeable ALGORITHM used immediately by the client to compute a result.

IMPORTANT INTERVIEW QUESTIONS

Q. How does the invoker (RemoteControl below) never know what a command actually does — it just calls execute()?
A. Because RemoteControl only depends on the Command interface, not on Light or any receiver — that's exactly what enables undo/redo and queuing: new command types can be dropped in without ever touching RemoteControl's code.

Q. How would you add undo support to this Command example?
A. Give Command an undo() method too; each concrete command stores the receiver's prior state (e.g. LightOnCommand remembers the light was previously off) and RemoteControl pops from its history stack, calling undo() instead of execute().

SENIOR-LEVEL DISCUSSION

The invoker never knows what a command actually does — it just calls execute(), which is what enables undo/redo and queuing. Relate this directly to a job queue or task scheduler if asked for a production example.

04 State

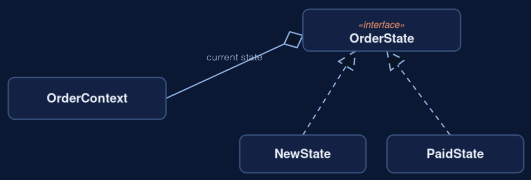
Behavioral

Definition: Lets an object alter its behavior when its internal state changes — the object appears to change its class from the outside.

Real-world use case: An OrderContext that behaves differently in NEW, PAID, and SHIPPED states, transitioning itself as events occur.

Spring Boot / real project: Order-processing or workflow engines in Spring Boot backends (an Order entity moving through NEW → PAID → SHIPPED → DELIVERED) are commonly modeled with State so each state class encapsulates its own valid transitions, instead of one giant service method full of switch-on-status logic.

UML Diagram



How to read it: OrderContext delegates behavior to the current OrderState object; each concrete state decides the NEXT state, so transition logic lives with the state itself, not the context.

Java Example

JAVA

```java
interface OrderState {
    void next(OrderContext ctx);
    String name();
}

class NewState implements OrderState {
    public void next(OrderContext ctx) { ctx.setState(new PaidState()); }
    public String name() { return "NEW"; }
}
class PaidState implements OrderState {
    public void next(OrderContext ctx) { ctx.setState(new ShippedState()); }
    public String name() { return "PAID"; }
}
class ShippedState implements OrderState {
    public void next(OrderContext ctx) { System.out.println("Already shipped"); }
    public String name() { return "SHIPPED"; }
}

class OrderContext {
    private OrderState state = new NewState();
    public void setState(OrderState state) { this.state = state; }
    public void advance() { state.next(this); System.out.println("Now: " + state.name()); }
}

// Usage
OrderContext order = new OrderContext();
order.advance(); // NEW -> PAID
order.advance(); // PAID -> SHIPPED
```

WHEN TO USE

- An object's behavior depends on its state and must change at runtime as state changes
- Code has large conditionals whose branches depend on the object's current state
- You want each state's transition rules encapsulated and easy to extend independently

WHEN NOT TO USE

- There are only two simple states (a boolean flag with an if/else is clearer than a full pattern)
- State transitions are trivial and unlikely to grow more complex over time

ADVANTAGES

- Eliminates large state-conditional blocks by localizing each state's behavior in its own class
- Makes adding a new state straightforward without touching existing state classes
- State transitions become explicit and easy to trace

DISADVANTAGES

- Increases the number of classes, one per state
- Can be overkill for objects with only two or three simple states

COMMON MISTAKES

- Confusing State with Strategy and letting the CLIENT choose transitions rather than the state classes managing them internally
- Leaving transition logic duplicated across the context AND the state classes instead of it living in exactly one place

DIFFERENCE: VS STRATEGY

Notice each state class decides the NEXT state itself — that self-transitioning behavior is exactly what separates State from Strategy, where the client picks the algorithm from outside and nothing transitions on its own.

IMPORTANT INTERVIEW QUESTIONS

Q. How would you prevent an illegal transition, e.g. going straight from NEW to SHIPPED?
A. Each state's next() method only knows how to transition to its own valid successor states — ShippedState.next() in this example intentionally does nothing (or throws), so illegal jumps are structurally impossible rather than checked with runtime if-statements scattered around.

Q. Is a Java enum with abstract methods per constant a valid State pattern implementation?
A. Yes — enums with per-constant method bodies (each enum constant overriding an abstract method) are a very common, concise way to implement State in Java when the full set of states is fixed and known at compile time.

SENIOR-LEVEL DISCUSSION

Notice each state class decides the NEXT state itself — that self-transitioning behavior is exactly what separates State from Strategy. A great follow-up is modeling this with a Java enum with abstract methods.

05 Template Method

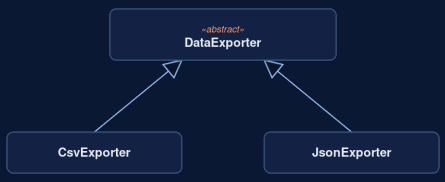
Behavioral

Definition: Defines the fixed skeleton of an algorithm in a base class, deferring specific, variable steps to subclasses without changing the overall structure.

Real-world use case: A DataExporter base class that fixes the order (fetch -> transform -> write) but lets CsvExporter and JsonExporter override just write().

Spring Boot / real project: JdbcTemplate and RestTemplate are the canonical Spring examples — the framework owns the fixed skeleton (open connection, execute, handle exceptions, close resources) and you plug in only the variable step via a callback (a RowMapper or a lambda).

UML Diagram



How to read it: export() in the base class is final and calls fetchData() -> transformData() -> writeData() in a fixed order; only writeData() is overridable.

Java Example

JAVA

```java
abstract class DataExporter {
    // Template method — defines the fixed algorithm skeleton
    public final void export() {
        fetchData();
        transformData();
        writeData();
    }
    private void fetchData() { System.out.println("Fetching data..."); }
    private void transformData() { System.out.println("Transforming data..."); }
    protected abstract void writeData(); // subclasses fill this in
}

class CsvExporter extends DataExporter {
    protected void writeData() { System.out.println("Writing as CSV"); }
}
class JsonExporter extends DataExporter {
    protected void writeData() { System.out.println("Writing as JSON"); }
}

// Usage
DataExporter exporter = new JsonExporter();
exporter.export();
```

WHEN TO USE

- Several classes implement an algorithm that's identical in structure but differs in a few steps
- You want to control the algorithm's overall shape while letting subclasses customize specific parts
- You want to enforce an invariant sequence of steps that subclasses cannot reorder

WHEN NOT TO USE

- The steps genuinely need to be reordered or skipped per subclass — that breaks the whole premise of a fixed template
- Composition-based alternatives (Strategy, passing in a callback/lambda) would be more flexible than committing to an inheritance hierarchy

ADVANTAGES

- Reuses common algorithm structure across subclasses while allowing targeted customization
- Enforces an invariant sequence of steps that subclasses can't accidentally break
- Follows the Hollywood Principle — 'don't call us, we'll call you' — subclasses only fill in blanks

DISADVANTAGES

- Relies on inheritance, which is a stronger, more rigid coupling than composition-based alternatives
- Can be harder to test individual steps in isolation since they're private methods of the base class

COMMON MISTAKES

- Forgetting to mark the template method final, letting subclasses accidentally override the fixed algorithm skeleton itself
- Reaching for inheritance-based Template Method when a simple higher-order function/callback (composition) would be more flexible in modern Java

DIFFERENCE: VS STRATEGY

Template Method uses INHERITANCE — subclasses fill in a few steps of a fixed algorithm defined in the base class. Strategy uses COMPOSITION — the whole algorithm is swapped out as one interchangeable object. When in doubt, composition (Strategy) is usually the more flexible, more testable choice in modern Java.

IMPORTANT INTERVIEW QUESTIONS

Q. Why should the template method itself be declared `final`?
A. To enforce the Hollywood Principle ('don't call us, we'll call you') and guarantee the fixed algorithm shape can't be overridden or reordered by a subclass — only the designated hook methods (like writeData()) are meant to vary.

Q. How do JdbcTemplate/RestTemplate embody this pattern?
A. Both classes own the fixed skeleton — acquiring a resource, executing the operation, handling exceptions, releasing the resource — and expose extension points (a RowMapper, a ResponseExtractor, or a lambda) for exactly the one step that legitimately varies per call, exactly like writeData() here.

SENIOR-LEVEL DISCUSSION

This is exactly how JdbcTemplate and RestTemplate work internally in Spring — the framework owns the algorithm's shape, you plug in the specifics. Contrast this cleanly against Strategy when asked 'inheritance vs composition'.

06 Chain of Responsibility

Behavioral

Definition: Passes a request along a chain of candidate handlers until one of them handles it, decoupling the sender from the specific receiver.

Real-world use case: A support-ticket system that tries L1Support, then L2Support, then Manager, stopping at whichever level can resolve the issue.

Spring Boot / real project: Servlet Filter chains and Spring Security's FilterChain (authentication, CSRF checks, authorization, logging, each as its own filter) are Chain of Responsibility in production — each filter decides to handle the request, short-circuit it, or pass it further down the chain.

UML Diagram



How to read it: Each handler holds a reference to the next handler in the chain and either resolves the request itself or forwards it along.

Java Example

JAVA

```java
abstract class SupportHandler {
    protected SupportHandler next;
    public void setNext(SupportHandler next) { this.next = next; }
    public abstract void handle(int severity);
}

class L1Support extends SupportHandler {
    public void handle(int severity) {
        if (severity <= 2) System.out.println("L1 resolved it");
        else if (next != null) next.handle(severity);
    }
}
class L2Support extends SupportHandler {
    public void handle(int severity) {
        if (severity <= 5) System.out.println("L2 resolved it");
        else if (next != null) next.handle(severity);
    }
}
class ManagerSupport extends SupportHandler {
    public void handle(int severity) { System.out.println("Manager resolved it"); }
}

// Usage
SupportHandler l1 = new L1Support();
SupportHandler l2 = new L2Support();
l1.setNext(l2);
l2.setNext(new ManagerSupport());
l1.handle(7); // escalates all the way to Manager
```

WHEN TO USE

- More than one object may handle a request, and the handler shouldn't be hard-coded ahead of time
- You want to issue a request without specifying the receiver explicitly
- The set of handlers and their order should be configurable/composable at runtime

WHEN NOT TO USE

- There's exactly one obvious handler for every request — direct dispatch is simpler
- You need EVERY handler to process the request rather than stopping at the first one that can (Observer fits that better)

ADVANTAGES

- Decouples sender from receiver — the client just sends the request into the chain
- Handler order and composition can change dynamically at runtime
- New handlers can be added without changing the sending code

DISADVANTAGES

- No guarantee a request will be handled at all unless the chain has a default/fallback handler
- Debugging can be harder — tracing which handler actually processed (or dropped) a request means following the whole chain

COMMON MISTAKES

- Forgetting a terminal/default handler, so requests that no one handles silently vanish
- Building a chain so long that performance and debuggability suffer — a chain isn't free indirection

DIFFERENCE: VS DECORATOR

Both link objects together and forward calls along a chain, but Decorator's every link ADDS behavior and always calls the next link. Chain of Responsibility's links can STOP the chain early once a handler resolves the request — the intent is selection/short-circuiting, not universal augmentation.

IMPORTANT INTERVIEW QUESTIONS

Q. How is this the shape behind servlet filter chains and Spring Security?
A. Each Filter (or Spring Security FilterChain link) decides to handle the request, short-circuit it (e.g., reject unauthenticated requests), or pass it on to filterChain.doFilter(...) — exactly the handle-or-forward decision each SupportHandler makes here.

Q. What happens if no handler in the chain can resolve the request?
A. Unless a default/catch-all handler is placed at the end of the chain, the request is silently dropped — a common production bug. Always design chains with an explicit terminal handler or a documented fallback behavior.

SENIOR-LEVEL DISCUSSION

This is the shape behind servlet filter chains and Spring Security's FilterChain — each link decides to handle the request or pass it on. Always mention the 'what if nobody handles it' failure mode.

07 Iterator

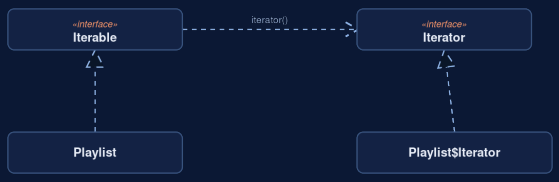
Behavioral

Definition: Provides a way to access the elements of a collection sequentially without exposing its underlying representation (array, list, tree, etc).

Real-world use case: Traversing a custom PlaylistCollection with a for-each loop exactly like you would an ArrayList, regardless of internal storage.

Spring Boot / real project: Spring Data's Page<T>/Slice<T> results and repository stream queries lean on the Iterator pattern to let you page through potentially huge result sets without loading everything into memory or caring how the underlying JDBC ResultSet cursor works.

UML Diagram



How to read it: Playlist exposes an Iterator without revealing that it's backed by an ArrayList internally — the traversal algorithm is decoupled from the storage structure.

Java Example

JAVA

```
class Playlist implements Iterable<String> {
    private List<String> songs = new ArrayList<>();
    public void addSong(String song) { songs.add(song); }

    @Override
    public Iterator<String> iterator() {
        return new Iterator<String>() {
            private int index = 0;
            public boolean hasNext() { return index < songs.size(); }
            public String next() { return songs.get(index++); }
        };
    }
}

// Usage
Playlist playlist = new Playlist();
playlist.addSong("Song A");
playlist.addSong("Song B");
for (String song : playlist) {
    System.out.println("Now playing: " + song);
}
```

WHEN TO USE

- You want to traverse a collection without exposing its internal representation
- You need multiple simultaneous, independent traversals over the same collection
- You want a uniform traversal interface across different aggregate structures (lists, trees, graphs)

WHEN NOT TO USE

- You're already using standard Java collections, which give you Iterator support for free — hand-rolling one is unnecessary
- Random access is required and sequential-only traversal doesn't fit the use case

ADVANTAGES

- Hides the aggregate's internal structure from client traversal code
- Supports multiple independent iterators over the same collection simultaneously
- Provides a uniform interface (for-each) regardless of the underlying storage

DISADVANTAGES

- For simple collections, it's extra ceremony over just exposing a getter for the backing structure
- Concurrent modification during iteration needs explicit handling (ConcurrentModificationException) to avoid subtle bugs

COMMON MISTAKES

- Modifying the backing collection directly while iterating over it, triggering a ConcurrentModificationException
- Reimplementing Iterator by hand for a structure that could just use java.util.Collection and get it for free

DIFFERENCE: VS VISITOR

Iterator externalizes TRAVERSAL — how to walk element by element. Visitor externalizes an OPERATION performed on each element once visited. They're complementary: you often iterate a structure and, at each step, apply a visitor's logic to the current element.

IMPORTANT INTERVIEW QUESTIONS

Q. Java's own Collection framework is built on this pattern — what does that mean concretely?
A. Every for-each loop under the hood calls iterator(), hasNext(), and next() on whatever Iterable the collection implements — List, Set, and Map's entrySet() all support this without the caller knowing whether the backing structure is an array, a linked list, or a hash table.

Q. What's the correct way to remove elements while iterating in Java?
A. Use the Iterator's own remove() method (or ListIterator's add/set/remove), not the collection's own remove() called mid-loop — calling the collection's remove() directly during iteration throws ConcurrentModificationException.

SENIOR-LEVEL DISCUSSION

Java's own Collection framework is built on this pattern — every for-each loop under the hood calls iterator(), hasNext(), and next(). Mention ConcurrentModificationException as the classic pitfall to watch for.

08 Mediator

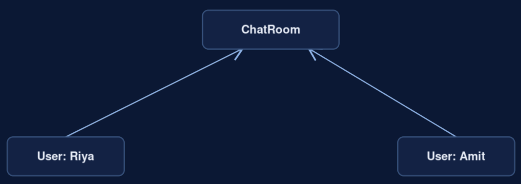
Behavioral

Definition: Centralizes complex communication between a set of objects into one mediator object, so those objects no longer refer to each other directly.

Real-world use case: A ChatRoom mediator through which every User sends messages, instead of every user holding references to every other user.

Spring Boot / real project: Spring's DispatcherServlet acts as a Mediator between incoming HTTP requests and the various controllers, view resolvers, and interceptors — none of those components talk to each other directly; they all coordinate through the central dispatcher.

UML Diagram



How to read it: Users never hold references to each other — every message is routed through the ChatRoom mediator, which decides how to broadcast it.

Java Example

JAVA

```java
class ChatRoom {
    public void showMessage(User user, String message) {
        System.out.println("[" + user.getName() + "]: " + message);
    }
}

class User {
    private String name;
    private ChatRoom room;
    User(String name, ChatRoom room) {
        this.name = name;
        this.room = room;
    }
    String getName() { return name; }
    void send(String message) {
        room.showMessage(this, message);
    }
}

// Usage
ChatRoom room = new ChatRoom();
User riya = new User("Riya", room);
User amit = new User("Amit", room);
riya.send("Hi Amit!");
amit.send("Hey Riya, ready for the interview?");
```

WHEN TO USE

- A set of objects communicate in complex, well-defined but many-to-many ways
- You want to reduce chaotic, tangled references between collaborating objects
- You want to reuse individual colleague objects independently of the specific others they talk to

WHEN NOT TO USE

- Only two objects need to communicate — a direct reference is simpler than a mediator
- The communication pattern is simple broadcast from one source (Observer is a better fit)

ADVANTAGES

- Reduces direct coupling between colleague objects to a single shared dependency (the mediator)
- Centralizes control logic, making it easier to change interaction rules in one place
- Colleague objects become simpler and more reusable in isolation

DISADVANTAGES

- The mediator itself can become a complex 'god object' if it absorbs too much logic
- Centralizing everything through one hub can become a bottleneck for very high-throughput interactions

COMMON MISTAKES

- Letting the Mediator accumulate unrelated responsibilities until it becomes an unmaintainable god class
- Introducing a Mediator for a simple two-object interaction where direct association would be far simpler

DIFFERENCE: VS OBSERVER

This reduces an O(n^2) web of direct object references down to one hub — it trades tight coupling for a single, more complex mediator class. Compare with Observer: Observer is a one-way broadcast from a subject; Mediator coordinates two-way, many-to-many interaction between peers.

IMPORTANT INTERVIEW QUESTIONS

Q. How does Mediator reduce an O(n^2) reference problem?
A. Without a mediator, N colleague objects that all need to talk to each other require up to N*(N-1) direct references. With a mediator, every colleague needs only ONE reference — to the mediator — collapsing the complexity to O(n) references, at the cost of centralizing logic into the mediator itself.

Q. Give a Spring-specific real-world example of Mediator.
A. DispatcherServlet is a Mediator between HTTP requests and Spring MVC's controllers, handler mappings, and view resolvers — none of those components reference each other directly; all coordination flows through the central dispatcher.

SENIOR-LEVEL DISCUSSION

This reduces an O(n^2) web of direct object references down to one hub — it trades tight coupling for a single, more complex mediator class. Be ready to sketch that math out on a whiteboard.

09 Memento

Behavioral

Definition: Captures and externalizes an object's internal state so it can be restored later, without violating the object's encapsulation.

Real-world use case: An undo feature in a document editor that snapshots TextEditor content before each edit, so it can roll back on request.

Spring Boot / real project: Workflow/form-wizard style features (a multi-step application form a user can save as a draft and resume later) often persist a Memento-shaped snapshot of form state to the database or session, letting the service restore exactly where the user left off without exposing the form entity's internals to the persistence layer.

UML Diagram



How to read it: Only the TextEditor (Originator) can read inside an EditorMemento; the Caretaker just stores mementos opaquely, without inspecting them.

Java Example

JAVA

```java
class EditorMemento {
    private final String content;
    EditorMemento(String content) { this.content = content; }
    String getContent() { return content; }
}

class TextEditor {
    private String content = "";
    void type(String text) { content += text; }
    EditorMemento save() { return new EditorMemento(content); }
    void restore(EditorMemento memento) { content = memento.getContent(); }
    String getContent() { return content; }
}

// Usage — caretaker keeps a history without inspecting it
TextEditor editor = new TextEditor();
editor.type("Hello ");
EditorMemento checkpoint = editor.save();
editor.type("World, this gets undone");
editor.restore(checkpoint);
System.out.println(editor.getContent()); // "Hello "
```

WHEN TO USE

- You need to implement undo/redo or checkpoint/rollback functionality
- You want to snapshot an object's state without exposing its internal fields publicly
- A direct interface to obtain state for rollback would violate the object's encapsulation

WHEN NOT TO USE

- The object's state is huge and snapshotting it repeatedly would be too memory/CPU expensive without extra optimization (e.g. diffs)
- Undo isn't actually a real requirement — don't build this speculatively

ADVANTAGES

- Preserves encapsulation — internal state is never exposed to the caretaker, only to the originator that created it
- Simplifies the originator by moving state-preservation logic into a dedicated object
- Makes undo/redo and checkpointing straightforward to implement correctly

DISADVANTAGES

- Storing many mementos can be memory-intensive for large objects or long undo histories
- Caretakers must be careful never to peek inside mementos, which requires discipline since Java doesn't enforce this as strictly as some languages

COMMON MISTAKES

- Letting the Caretaker read or modify memento internals, breaking the encapsulation the pattern is meant to protect
- Storing full deep copies of large object graphs on every single change instead of considering incremental/diff-based snapshots for large state

DIFFERENCE: VS COMMAND

Memento supports undo by SNAPSHOTTING STATE before a change and restoring it later. Command supports undo differently — by INVERTING THE OPERATION itself (calling an undo() on the command). Memento is simpler when full-state snapshots are cheap; Command's approach is more memory-efficient for large state but requires each action to know how to reverse itself.

IMPORTANT INTERVIEW QUESTIONS

Q. Why does only the Originator get to read inside a Memento?
A. The Caretaker (history stack) stores mementos but never looks inside them — only the Originator that created a memento can read it back, via a narrow interface it controls itself. This preserves the Originator's encapsulation: its internal representation is never exposed to unrelated code.

Q. How would you limit memory growth for a long-running undo history?
A. Cap the history size (e.g. keep only the last 50 checkpoints, evicting the oldest), or move to incremental/diff-based mementos that store just the delta from the previous snapshot rather than full state every time.

SENIOR-LEVEL DISCUSSION

The Caretaker (history stack) stores mementos but never looks inside them — only the Originator that created a memento can read it back. Undo stacks in editors and IDEs are the go-to production example.

10 Visitor

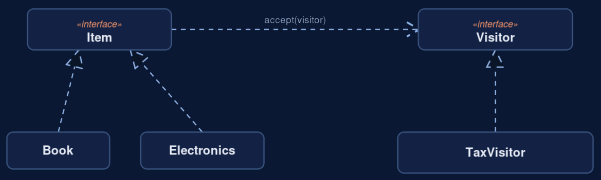
Behavioral

Definition: Separates an algorithm from the object structure it operates on, letting you add new operations without modifying the existing classes (double dispatch).

Real-world use case: A tax calculator that visits Book, Electronics, and Food items in a shopping cart, applying a different tax rule per type without editing those classes.

Spring Boot / real project: AST-processing tools, code generators, and export pipelines (e.g. a ReportVisitor that walks a hierarchy of Section/Chart/Table elements and produces PDF vs Excel output per element type) reach for Visitor in Spring Boot batch/reporting services when new export formats are added more often than new element types.

UML Diagram



How to read it: accept(visitor) picks the concrete Item class; the resulting visit(this) call picks the right overload — that two-step is 'double dispatch'.

Java Example

JAVA

```java
interface Visitor {
    void visit(Book book);
    void visit(Electronics electronics);
}

interface Item {
    void accept(Visitor visitor);
}

class Book implements Item {
    double price = 500;
    public void accept(Visitor visitor) { visitor.visit(this); }
}
class Electronics implements Item {
    double price = 15000;
    public void accept(Visitor visitor) { visitor.visit(this); }
}

class TaxVisitor implements Visitor {
    public void visit(Book book) {
        System.out.println("Book tax: " + (book.price * 0.0)); // exempt
    }
    public void visit(Electronics e) {
        System.out.println("Electronics tax: " + (e.price * 0.18));
    }
}

// Usage
List<Item> cart = List.of(new Book(), new Electronics());
Visitor taxVisitor = new TaxVisitor();
for (Item item : cart) item.accept(taxVisitor);
```

WHEN TO USE

- You need to perform several unrelated operations across a stable object structure, and new operations are added more often than new element types
- You want to keep related behavior for an operation together in one class instead of scattered across many element classes
- You're implementing something like an AST walker, exporter, or a tax/pricing rules engine over fixed item types

WHEN NOT TO USE

- The set of element types (classes being visited) changes frequently — every new type forces you to update every existing Visitor implementation
- There's only ever going to be one operation — a simple method on the element classes themselves is far simpler

ADVANTAGES

- New operations can be added without modifying the visited classes (Open/Closed for new operations)
- Groups related behavior for one operation together in a single Visitor class, rather than scattering it across element classes

DISADVANTAGES

- Adding a NEW element type requires updating every existing Visitor implementation — the trade-off is exactly inverted from adding operations
- Breaks encapsulation somewhat, since visitors often need access to element internals to do their work

COMMON MISTAKES

- Using Visitor when element types change often — that's the one scenario it handles worst
- Forgetting to overload visit() for every concrete element type, causing silent double-dispatch failures when a new type is added but a Visitor isn't updated

DIFFERENCE: VS STRATEGY

Visitor operates across a whole FAMILY of related element types via double dispatch, applying one operation differently per type. Strategy swaps ONE algorithm used by ONE context. Visitor is the right call when you have many types and want to add cross-cutting operations without touching those type classes.

IMPORTANT INTERVIEW QUESTIONS

Q. Explain 'double dispatch' precisely.
A. accept(visitor) is called on the element, which resolves the CONCRETE element class (first dispatch, based on the element's runtime type) and then calls visitor.visit(this) with that concrete type, which resolves the correct overload of visit() (second dispatch, based on the argument's compile-time type from within accept()) — together these two dispatches let the right combination of (element type, operation) run without any instanceof checks.

Q. Why does adding a new Item type ripple through every Visitor?
A. Because every concrete Visitor implementation must add a new visit(NewType) overload to handle it — if any Visitor implementation is missed, that operation silently doesn't support the new type. This is the core trade-off versus just adding a method to the element class itself.

SENIOR-LEVEL DISCUSSION

The 'double dispatch' trick — accept(visitor) picks the class, then visit(this) picks the overload — is exactly what makes Visitor work in Java. Be ready to explain why it inverts the usual Open/Closed trade-off (easy new operations, hard new types).

11 Interpreter

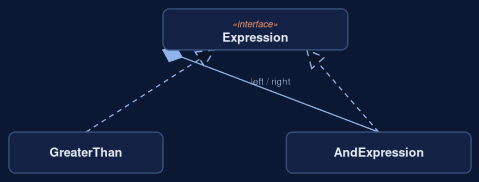
Behavioral

Definition: Given a language, defines a representation for its grammar along with an interpreter that evaluates sentences in that language.

Real-world use case: A rules engine that parses simple boolean expressions like 'age > 18 AND hasLicense' into an executable condition tree.

Spring Boot / real project: Spring Expression Language (SpEL, used in `@Value("#{...}")` and Spring Security's `@PreAuthorize("hasRole('ADMIN')")`) is a real production Interpreter — each SpEL expression is parsed into an executable tree of expression nodes, exactly like the GreaterThan/AndExpression classes below.

UML Diagram



How to read it: AndExpression is itself composed of two Expression children (Composite-style), and interpret() recursively evaluates the whole tree.

Java Example

JAVA

```
interface Expression {
    boolean interpret(int age);
}

class GreaterThan implements Expression {
    private int value;
    GreaterThan(int value) { this.value = value; }
    public boolean interpret(int age) { return age > value; }
}

class AndExpression implements Expression {
    private Expression left, right;
    AndExpression(Expression left, Expression right) {
        this.left = left; this.right = right;
    }
    public boolean interpret(int age) {
        return left.interpret(age) && right.interpret(age);
    }
}

// Usage — build "age > 18 AND age > 0" as an expression tree
Expression isAdult = new GreaterThan(18);
Expression isPositive = new GreaterThan(0);
Expression rule = new AndExpression(isAdult, isPositive);
System.out.println(rule.interpret(21)); // true
```

WHEN TO USE

- You have a simple, well-defined grammar and need to evaluate sentences in that grammar repeatedly
- The grammar is small enough that a class-per-rule hierarchy stays manageable
- Efficiency of parsing/evaluation isn't the top priority — clarity of the grammar model is

WHEN NOT TO USE

- The grammar is even moderately complex — a proper parser generator or expression library will scale far better than hand-rolled classes
- Performance-critical evaluation is required — real parser/interpreter libraries are heavily optimized compared to a naive tree of classes

ADVANTAGES

- Makes the grammar directly visible and extensible as a class hierarchy
- Easy to add new expression types by adding new classes implementing the interface
- Each grammar rule is isolated, self-contained, and independently testable

DISADVANTAGES

- Scales poorly — complex grammars produce a large, unwieldy class hierarchy
- Rarely the right engineering choice today versus mature parsing/expression libraries

COMMON MISTAKES

- Hand-rolling Interpreter for anything beyond a toy grammar, instead of reaching for a real parser generator (ANTLR) or an existing expression library (SpEL, MVEL)
- Letting the expression tree grow so deep that recursive interpret() calls risk stack overflow on pathological input

DIFFERENCE: VS COMPOSITE

Interpreter is essentially Composite applied to a grammar: expression nodes compose into a tree exactly like Composite's part-whole hierarchy, but each node's operation is specifically interpret() — evaluating itself against some context, rather than a general-purpose operation.

IMPORTANT INTERVIEW QUESTIONS

Q. Would you actually hand-write an Interpreter in a production Spring Boot service today?
A. Rarely — for anything beyond a toy grammar, reach for a parser generator like ANTLR or an existing expression engine like Spring Expression Language (SpEL) or MVEL rather than hand-rolling a class hierarchy; interviewers mainly want to see you can model a grammar as a class hierarchy conceptually.

Q. How does Interpreter relate to Composite?
A. Structurally identical — expression nodes are composed into a tree just like Composite's part-whole hierarchy — but Interpreter's specific intent is that each node's shared operation is interpret(), evaluating itself against a context (like the 'age' value here) rather than a generic operation like getSize().

SENIOR-LEVEL DISCUSSION

Rarely hand-rolled today (regex/parser libraries and SpEL usually win), but interviewers like it to test whether you can model a grammar as a class hierarchy — and whether you know when NOT to use it in production.

Quick Reference

All 23 patterns at a glance — use this as a last-minute revision sheet before your interview.

	Category	One-Line Purpose
Singleton	Creational	Ensures a class has exactly one instance across the entire application and provides a single, well-known global access point to it.
Factory Method	Creational	Defines an interface for creating an object, but lets subclasses decide which concrete class to instantiate. The client only ever depends on the abstract product.
Abstract Factory	Creational	Produces families of related or dependent objects without specifying their concrete classes — one factory interface, multiple coordinated products.
Builder	Creational	Separates the construction of a complex object from its representation, assembling it step by step through a fluent, readable API.
Prototype	Creational	Creates new objects by cloning an existing instance ('the prototype') rather than instantiating from scratch — useful when creation is expensive.
Adapter	Structural	Converts the interface of one class into another interface that a client expects, allowing otherwise-incompatible classes to work together.
Bridge	Structural	Decouples an abstraction from its implementation so the two class hierarchies can vary and evolve independently of each other.
Composite	Structural	Composes objects into tree structures to represent part-whole hierarchies, letting clients treat individual objects and compositions uniformly.
Decorator	Structural	Attaches new responsibilities to an object dynamically at runtime by wrapping it, as a flexible alternative to subclassing.
Facade	Structural	Provides a simplified, unified interface to a larger and more complex subsystem, hiding its internal wiring from the client.
Flyweight	Structural	Minimizes memory usage by sharing as much data as possible with other similar objects — separates intrinsic (shared) from extrinsic (per-instance) state.
Proxy	Structural	Provides a placeholder or surrogate for another object to control access to it — for lazy loading, security checks, logging, or caching.
Strategy	Behavioral	Defines a family of interchangeable algorithms, encapsulates each one, and lets the client swap between them at runtime.
Observer	Behavioral	Defines a one-to-many dependency so that when one object (the subject) changes state, all its dependents (observers) are notified automatically.
Command	Behavioral	Encapsulates a request (and everything needed to execute it) as an object, enabling queuing, logging, undo/redo, and decoupled invokers.
State	Behavioral	Lets an object alter its behavior when its internal state changes — the object appears to change its class from the outside.
Template Method	Behavioral	Defines the fixed skeleton of an algorithm in a base class, deferring specific, variable steps to subclasses without changing the overall structure.
Chain of Responsibility	Behavioral	Passes a request along a chain of candidate handlers until one of them handles it, decoupling the sender from the specific receiver.
Iterator	Behavioral	Provides a way to access the elements of a collection sequentially without exposing its underlying representation (array, list, tree, etc).
Mediator	Behavioral	Centralizes complex communication between a set of objects into one mediator object, so those objects no longer refer to each other directly.
Memento	Behavioral	Captures and externalizes an object's internal state so it can be restored later, without violating the object's encapsulation.
Visitor	Behavioral	Separates an algorithm from the object structure it operates on, letting you add new operations without modifying the existing classes (double dispatch).
Interpreter	Behavioral	Given a language, defines a representation for its grammar along with an interpreter that evaluates sentences in that language.

Interview Q&A

Common interview questions — rehearse the trade-offs, not just the definitions.

Q. Where have you actually used design patterns in a Spring Boot project?
A. Talk concretely: Singleton for Spring beans, Strategy for pluggable pricing/discount rules, Factory Method for creating notification senders, Proxy via Spring AOP for @Transactional and logging, Template Method inside JdbcTemplate / RestTemplate, and Observer via Spring's ApplicationEventPublisher.

Q. Factory Method vs Abstract Factory — what's the difference?
A. Factory Method creates one product via a single overridden method. Abstract Factory creates a whole family of related products through multiple factory methods on one interface.

Q. Why is Singleton considered an anti-pattern by some?
A. It introduces global state, can make unit testing harder through hidden dependencies, and can hide coupling. In Spring, prefer container-managed singleton scope and dependency injection.

Q. How does Decorator differ from Proxy?
A. Both can wrap an object behind the same interface. Decorator adds responsibilities or behavior; Proxy controls access to the object, such as lazy loading, security, logging, or caching.

Q. Strategy vs State — aren't they structurally the same?
A. The structures can look nearly identical, but intent differs: Strategy is chosen by the client from outside; State transitions internally as the object's logic reacts to events.

Q. When would you reach for Composite?
A. When you have a part-whole tree and want uniform treatment of leaves and containers — file systems, UI component trees, and org-chart hierarchies are classic examples.

Q. What's the real difference between Adapter and Facade?
A. Adapter makes one existing interface compatible with what a client expects. Facade creates a simplified interface over a whole subsystem of multiple classes.

Q. How do you decide whether a design pattern is actually justified?
A. Start from the recurring design problem and the variation you need to isolate. Explain the trade-off: the pattern should reduce coupling, duplication, or change impact enough to justify its extra abstraction.